

Lava Deployment Guide

This guide covers building and deploying the three Lava artifacts:

1. **lava-api-go** — the Go API service (OCI image + binary).
2. **Android client app** (:app, digital.vasic.lava.client).
3. **On-device API app** (:api-app, digital.vasic.lava.api).

Everything here is derived from the actual scripts/files in the repo: build_and_release.sh, lava-api-go/Makefile, lava-api-go/docker/Dockerfile, docker-compose.yml, start.sh / stop.sh, and scripts/firebase-distribute.sh. Where a step is not backed by a script flag, it is marked UNCONFIRMED:.

Local-Only CI/CD. Lava forbids hosted CI. Every build/test/release step runs on a developer machine or a self-hosted local runner. See the root CLAUDE.md "Local-Only CI/CD" section.

1. Prerequisites

- A rootless container runtime: **Podman** (preferred) or **Docker** (build_and_release.sh:123-136 auto-detects either; start.sh delegates to tools/lava-containers, which auto-detects). No sudo (root CLAUDE.md §6.U).
- Go toolchain (the Dockerfile build stage uses golang:1.26-alpine, Dockerfile:24; lava-api-go/CLAUDE.md states Go 1.24+).
- JDK + Android SDK for the Gradle builds (the ./gradlew wrapper drives them).
- A repo-root .env (gitignored) — see §6. .env.example documents the keys.

2. lava-api-go — build

2.1 Makefile targets

From [lava-api-go/Makefile](#):

Target	Command it runs	Purpose
make all	build + test	Default.

Target	Command it runs	Purpose
make build	go build -o bin/lava-api-go ./cmd/lava-api-go and go build -o bin/healthprobe ./cmd/healthprobe	Build the two binaries.
make test	go test -race -count=1 ./...	Unit/contract/e2e/parity (no external deps).
make vet	GOMAXPROCS=2 go vet ./...	Built-in correctness analyzer.
make lint	./scripts/golangci-lint.sh	golangci-lint (containerized; exit 3 = tool unavailable).
make cover	GOMAXPROCS=2 go test ./... -coverprofile=coverage.out -covermode=atomic then go tool cover -func	Coverage rollup (real-Postgres integration tests are NOT in this default rollup).
make ci	./scripts/ci.sh	Local CI entry point.
make generate	./scripts/generate.sh	Regenerate oapi-codegen output.
make migrate-up	./scripts/migrate.sh up	Apply migrations.
make migrate-down	./scripts/migrate.sh down	Roll back migrations.
make image	docker build -f docker/Dockerfile -t lava-api-go:dev .	Build the OCI image (dev tag).
make clean	rm -rf bin/ coverage.* *.out	Clean.

Per lava-api-go/CLAUDE.md §6.K, the constitutional **gate** build of a release artifact runs through the vasic-digital/Containers orchestration; the host make build / make image are permitted for iteration. The release-artifact binary build done by build_and_release.sh (§5) uses CGO_ENABLED=0 go build -trimpath -ldflags='-s -w' (build_and_release.sh:74).

2.2 OCI image (multi-stage Dockerfile)

From [lava-api-go/docker/Dockerfile](#), build context is the **project root**:

Stage	Base	Output
build	golang:1.26-alpine	statically links lava-api-go + healthprobe (CGO_ENABLED=0, GOOS=linux, GOARCH=amd64).
migrate	golang:1.26-alpine	installs golang-migrate; ENTRYPOINT runs migrate -path ../migrations -database "\$LAVA_API_PG_URL" up (shell-form so \$LAVA_API_PG_URL expands).
runtime	gcr.io/distroless/static-debian12:nonroot	ships both binaries; EXPOSE 8443/udp 8443/tcp; HEALTHCHECK --interval=10s --retries=6 CMD ["/usr/local/bin/healthprobe"]; ENTRYPOINT ["/usr/local/bin/lava-api-go"].

The runtime image is distroless (no /bin/sh), which is why the HEALTHCHECK uses exec/JSON-array form. Build with --format=docker (not OCI) so the HEALTHCHECK survives in the image config — build_and_release.sh and start.sh both export BUILDAH_FORMAT=docker for exactly this reason (build_and_release.sh:100, start.sh:114; forensic anchor 2026-04-29 in both).

3. lava-api-go — run

3.1 docker-compose profiles

[docker-compose.yml](#) defines these profiles:

Profile	Services
api-go (default)	lava-postgres (postgres:16-alpine) → lava-migrate (one-shot) → lava-api-go
observability	Prometheus (v2.51.0), Loki (2.9.6), Promtail (2.9.6), Tempo (2.4.0), Grafana (10.4.2)
dev-docs	Swagger UI (v5.17.14) serving api/openapi.yaml at 127.0.0.1:8081

Service wiring:

- lava-postgres — DB lava_api, user lava, password from \$ {LAVA_PG_PASSWORD} (required); published to 127.0.0.1:8432 → container 5432; healthcheck pg_isready -U lava -d lava_api.
- lava-migrate — built from the Dockerfile migrate stage; depends on lava-postgres healthy; runs migrations against postgres://

```
lava:${LAVA_PG_PASSWORD}@lava-postgres:5432/lava_api?
sslmode=disable; restart: "no" (one-shot).
```

- lava-api-go — built from the Dockerfile runtime stage; depends on lava-migrate completing successfully; **network_mode: host** so mDNS reaches the LAN. Key env (docker-compose.yml:64-83):
LAVA_API_PG_URL, LAVA_API_LISTEN=":8443",
LAVA_API_METRICS_LISTEN=":9091", LAVA_API_TLS_CERT /
LAVA_API_TLS_KEY (mounted from ./lava-api-go/docker/tls),
LAVA_API_OTLP_ENDPOINT, and the LAVA_AUTH_* set (field name, HMAC secret, active/retired client lists, min-supported version, trusted proxies). No compose healthcheck: block — the Dockerfile's HEALTHCHECK is relied on (the runtime image is sh-less; see the comment at docker-compose.yml:86-98).

Do not bring the compose file up directly (docker-compose.yml:19-20). Orchestration is owned by tools/lava-containers.

3.2 start.sh / stop.sh (the supported entry point)

`start.sh` delegates to the Lava-domain CLI tools/lava-containers/bin/lava-containers (building it on first run, `start.sh:70-76`). It also: provisions TLS material via `lava-api-go/scripts/gen-cert.sh` if absent (`start.sh:79-81`); appends a default `LAVA_PG_PASSWORD` to `.env` if missing (`start.sh:86-89`); and exports `BUILDAH_FORMAT=docker` (`start.sh:114`).

```
./start.sh                                # profile=api-go
    (default)
./start.sh --with-observability            # + observability
    profile
./start.sh --dev-docs                     # + Swagger UI (dev-
    docs profile)
./start.sh --with-observability --dev-docs # combine
./stop.sh                                 # stop + remove the
    container
```

Flags map 1:1 onto the `lava-containers -profile / -with-observability / -dev-docs` flags (`start.sh:57-58`). Runtime status:

```
./tools/lava-containers/bin/lava-containers -cmd=status #
    runtime, health, advertised IPs
```

(from root `CLAUDE.md` Commands section).

3.3 Migrations

Applied automatically by the `lava-migrate` one-shot service before `lava-api-go` starts (§3.1). Manually: `make migrate-up / make migrate-down` (`Makefile:35-39`, wrapping `scripts/migrate.sh`). Migration SQL lives in `lava-api-go/migrations/0001..0009`; see [docs/db/schema.md](#).

4. Android client app (:app) — build, sign, distribute

4.1 Identity & signing

From app/build.gradle.kts:

- `applicationId = "digital.vasic.lava.client";` debug build adds `applicationIdSuffix = ".dev"`.
- `versionName / versionCode` are declared in app/build.gradle.kts (read by build_and_release.sh:21-22 and scripts/firebase-distribute.sh).
- Signing configs read keystore password + dir from .env (KEYSTORE_PASSWORD, KEYSTORE_ROOT_DIR, default keystores): keystores/debug.keystore (alias debug) and keystores/release.keystore (alias release). The keystores/ dir is gitignored (§6.H).
- Release build: `isMinifyEnabled = true, isObfuscate = false` (root CLAUDE.md "Release build quirks"); debug build `isMinifyEnabled = false`.
- The build also expects app/google-services.json (gitignored) for Firebase.

Direct Gradle builds (root CLAUDE.md Commands):

```
./gradlew :app:assembleDebug      # debug APK  
                                (applicationIdSuffix .dev)  
./gradlew :app:assembleRelease    # release APK (signed via  
                                keystores/)
```

4.2 build_and_release.sh

build_and_release.sh builds all artifacts and copies them into releases/:

1. `./gradlew clean`
2. `:app:assembleDebug, :app:assembleRelease`
3. `:api-app:assembleDebug, :api-app:assembleRelease`
4. `lava-api-go` static binary (`CGO_ENABLED=0 go build -trimpath -ldflags='-s -w'`)
5. Saves the `lava-api-go` OCI image tarball (docker-format) if a runtime exists.

Outputs:

- `releases/<APP_VERSION>/android-debug/digital.vasic.lava.client-<ver>-debug.apk`
- `releases/<APP_VERSION>/android-release/digital.vasic.lava.client-<ver>-release.apk`
- `releases/<APP_VERSION>/api-go/lava-api-go-<APIGO_VERSION> (+ .image.tar if built)`

- releases/api-app/<API_APP_VERSION>/android-{debug,release}/digital.vasic.lava.api-<ver>-{debug,release}.apk

4.3 Firebase App Distribution — two-stage (§6.AA)

`scripts/firebase-distribute.sh` uploads built APKs to Firebase App Distribution. The default mode is **debug-only** (Stage 1); release (Stage 2) must follow.

```
./scripts/firebase-distribute.sh                # default:
    debug only (MODE=debug)
./scripts/firebase-distribute.sh --debug-only    # Stage 1:
    debug APK
./scripts/firebase-distribute.sh --release-only  # Stage 2:
    release APK (requires Stage 1 first)
./scripts/firebase-distribute.sh --debug-and-release # legacy
    combined (NOT recommended)
./scripts/firebase-distribute.sh --release-notes "<text>" #
    custom notes
./scripts/firebase-distribute.sh --app api-app    # distribute
    the on-device API app instead
```

(Flags from `firebase-distribute.sh`:52-61. `--app` is `client` (default) or `api-app`, `firebase-distribute.sh`:80-106.)

The script enforces these gates **before** uploading (refuses on failure):

- **§6.P Gate 1** — `versionCode` strictly greater than the last distributed code for the channel (per-channel pointers `last-version-debug` / `last-version-release`, `firebase-distribute.sh`:167-196).
- **§6.AA Gate** — `--release-only` requires the debug stage to have already distributed this `versionCode` (`firebase-distribute.sh`:204-214).
- **§6.P Gate 2** — `CHANGELOG.md` contains an entry for this version (`firebase-distribute.sh`:217-221).
- **§6.P Gate 3** — a per-version snapshot file exists at `.lava-ci-evidence/distribute-changelog/<channel>/<ver>-<code>.md` (`firebase-distribute.sh`:223-229); shipped as the release notes.
- **Phase 1 Gates 4+5** (client only) — fresh `LAVA_AUTH_OBFUSCATION_PEPPER` (no reuse across releases) and `LAVA_AUTH_CURRENT_CLIENT_NAME == android-<ver>-<code>` present in `LAVA_AUTH_ACTIVE_CLIENTS` (`firebase-distribute.sh`:261-306).

It then calls `firebase appdistribution:distribute <apk> --app <app-id> --project <project> --testers <testers> --release-notes <notes>` (`firebase-distribute.sh`:396-401). Firebase config (token, project id, app ids, tester list) comes from `.env` via `scripts/firebase-env.sh`.

§6.Z requires the corresponding tests/Challenges to have been **executed** against the exact artifact before distributing; §6.AA requires operator real-device verification of the Stage-1 debug build before Stage 2. See root CLAUDE.md §6.Z / §6.AA. A manual sideload of a built APK is possible via `adb install <path-to.apk>` (standard Android tooling), but the supported distribution channel is Firebase. UNCONFIRMED: no `adb install` wrapper exists in the repo scripts.

5. On-device API app (:api-app) — build & distribute

The on-device API server app runs the lava-api-go engine in-process on an Android device (the embed router, lava-api-go/internal/router/router.go:6).

From api-app/build.gradle.kts:

- `applicationId = "digital.vasic.lava.api"` (namespace lava.api.app); debug build adds `applicationIdSuffix = ".dev"`.
- Same `.env`-driven signing as :app (keystores/debug.keystore, keystores/release.keystore).

Build (via `build_and_release.sh`, §4.2, or directly):

```
./gradlew :api-app:assembleDebug
./gradlew :api-app:assembleRelease
```

Distribute via the same script with `--app api-app` (`firebase-distribute.sh --app api-app`). The api-app uses a separate Firebase channel (`firebase-app-distribution-api-app`) and separate app-id env vars (`LAVA_FIREBASE_API_APP_DEV_APP_ID` / `LAVA_FIREBASE_API_APP_ID`, `firebase-distribute.sh`:96-97). Phase 1 client-auth gates (pepper / client name) are **skipped** for the api-app (it is the server side of the auth scheme, `firebase-distribute.sh`:252-262).

6. Configuration (.env)

Signing + Firebase + service config come from a gitignored repo-root `.env` (`.env.example` documents the keys). Notable keys referenced by the scripts and compose file:

Key	Used by	Notes
KEYSTORE_PASSWORD, KEYSTORE_ROOT_DIR		Default keystores.

Key	Used by	Notes
	app / api-app Gradle signing	
LAVA_PG_PASSWORD	lava- postgres / lava- migrate / lava-api-go compose	Required; start.sh writes a LAN default if absent.
LAVA_AUTH_FIELD_NAME	lava-api-go auth middleware	The Lava- Auth header name (not hardcoded, §6.R).
LAVA_AUTH_HMAC_SECRET, LAVA_AUTH_ACTIVE_CLIENTS, LAVA_AUTH_RETIRED_CLIENTS	auth middleware	Client- attestation allowlists.
LAVA_AUTH_MIN_SUPPORTED_VERSION_NAME / _CODE	auth middleware	Returned in the 426 body (defaults 1.2.6 / 1026).
LAVA_AUTH_OBFUSCATION_PEPPER, LAVA_AUTH_CURRENT_CLIENT_NAME	firebase- distribute.sh Gates 4/5	Client APK auth rotation.
LAVA_FIREBASE_TOKEN, LAVA_FIREBASE_PROJECT_ID, LAVA_FIREBASE_*_APP_ID, LAVA_FIREBASE_TESTERS	firebase- distribute.sh	Distribution. Tokens never echoed (§6.H).

Never commit .env, keystores/, or app/google-services.json (root CLAUDE.md §6.H; pre-push rejects).

7. Release tagging

scripts/tag.sh tags each app/service as Lava-<App>-<versionName>-<versionCode> and refuses to operate without local CI evidence + the §6.I/§6.AE real-device attestation (root CLAUDE.md "Release tagging gate"). The lava-api-go version lives in lava-api-go/internal/version/version.go (Name / Code); the Android versions in the respective build.gradle.kts.

8. Cross-references

- API reference: [docs/api/README.md](#)
- Database schema: [docs/db/schema.md](#)
- Firebase distribution detail: [docs/FIREBASE.md](#)
- Architecture: [docs/ARCHITECTURE.md](#)
- Build & release script: [build_and_release.sh](#)
- Container CLI glue: [tools/lava-containers/](#) (driven by `start.sh` / `stop.sh`)